

PBL Secondary Research Report

**TOPIC: "FPGA Implementation of Rate 1/2
Convolutional Encoder for UART Based Serial Data
Transmission"**

NAME	USN
Ibrahim Affan	4SO24EC046
Kaushik N(Leader)	4SO24EC058
Kruthi Kundalpady	4SO24EC060
Lereina Reemal Pashan	4SO24EC061

PBL Secondary Research Report

Introduction

Overview

Digital communication systems like disaster management and satellite communication send data over noisy channels, which can cause errors. To reduce these errors, Forward Error Correction (FEC) adds extra bits so the receiver can detect and correct mistakes.

In this project, a convolutional encoder is implemented on an FPGA to encode a continuous stream of data. It processes one bit per clock cycle at 50 MHz using simple XOR logic, enabling fast and reliable communication for real-time applications.

Problem Statement

Reliable digital communication is critical for disaster management systems, satellite communication, and rural connectivity. Error-control coding improves transmission reliability but increases computational overhead. Develop a hardware-based solution on FPGA that enhances the reliability of digital data transmission over a noisy channel.

Objectives

- Design a rate 1/2 convolutional encoder ($K = 3$) on FPGA.
- Use a UART module to take serial input data.
- Implement the encoder using a shift register and XOR logic.
- Generate output using given polynomials ($g_1 = 111$, $g_2 = 101$).
- Convert the two output bits into a serial output stream.
- Test the design using simulation (Vivado).

Literature Review

2.1 Review of Existing Work

Several research works have been carried out on the design and FPGA implementation of convolutional encoders and Viterbi decoders, mainly focusing on improving performance, area, and reliability.

- Paper 1**
 This work uses a Radix-2 full-parallel architecture with constraint length $K=7$, which is commonly used in wireless standards. The design achieves a high throughput of around 80 Mbps, making it suitable for high-speed applications. However, the system introduces high decoding latency and requires a complex memory structure, which increases design complexity.
- Paper 2**
 This paper presents an adaptive Viterbi decoder with constraint length $K=3$, where the traceback length can be adjusted based on signal conditions. This improves flexibility and allows better control over performance. However, due to the low constraint length, the error correction capability is limited. Also, the paper does not provide clear throughput values, making it difficult to compare with other designs.
- Paper 3**
 This work implements a standard Viterbi decoder ($K=7$) using Verilog HDL. The design is functionally correct and suitable for FPGA implementation. However, the paper mainly focuses on functionality and does not provide details about speed, throughput, or power consumption, which are important for evaluating performance.
- Paper 4**
 This paper focuses on an area-optimized design using $K=7$ targeting FPGA implementation. The encoder uses very minimal resources, but the decoder consumes a large amount of hardware resources, especially memory and logic units. Additionally, the paper does not provide throughput or power analysis, limiting its practical evaluation.

2.2 Comparative Analysis

Paper	Architecture	Throughput	Area (Resource Usage)	Limitation
Paper 1 (Sun & Ding, 2012)	Radix-2, Full-parallel ACS, $K=7$	80 Mbps	~316 slices, 510 LUTs	High latency, complex memory design
Paper 2 (Wong et al., 2009)	Radix-2, Adaptive Viterbi, $K=3$	Not specified	High (~866 slices)	Low error correction, no throughput data
Paper 3 (Kavinilavu et al., 2011)	Standard Viterbi, $K=7$	Not specified	Moderate (~325 slices encoder)	No speed/power analysis, high I/O usage

Paper 4 (Soreng & Kumar, 2013)	Area-optimized VLSI, K=7	Not specified	Very high (decoder ~7991 ALUTs)	Large decoder area, no throughput/power data
--------------------------------	--------------------------	---------------	---------------------------------	--

2.3 Research Gap Identification

- Most existing designs optimize either speed or area, but not both together.
- Power consumption analysis is missing in most of the works.
- Several papers do not report throughput, which is important for performance evaluation.
- Some designs use low constraint length ($K=3$), which is not suitable for modern communication systems.
- There is a need for a design that is balanced, efficient, and practical for FPGA implementation.

Proposed System Design

Design justification

Non-Systematic Feedforward Convolutional Encoder: A non-systematic encoder is used so that both output bits are generated using XOR combinations of current and previous bits, which improves error protection by not transmitting the input bit directly.

Constraint Length ($K = 3$): A constraint length of 3 is chosen to maintain a simple design with low hardware usage while still providing effective error correction.

Code Rate ($1/2$): A code rate of $1/2$ is selected to ensure a good balance between error correction capability and bandwidth efficiency.

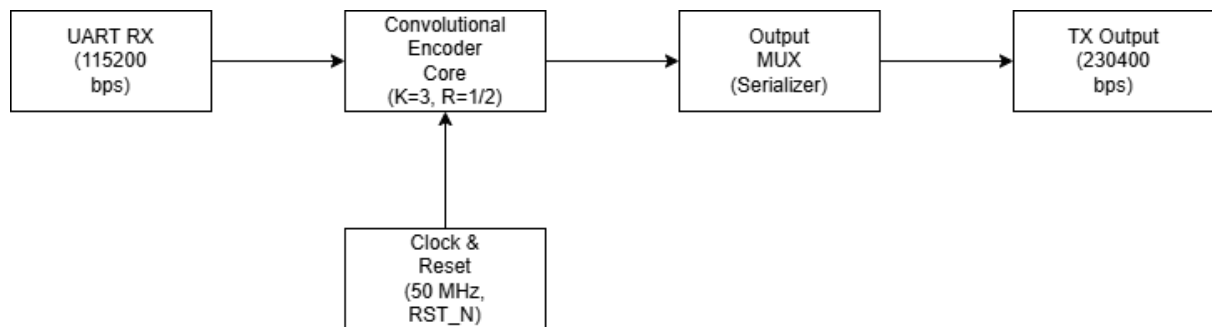
Generator Polynomials: These polynomials are used because they are standard and provide strong error detection and correction performance.

FPGA Implementation: FPGA is used to achieve high-speed, real-time processing with low latency and parallel operation.

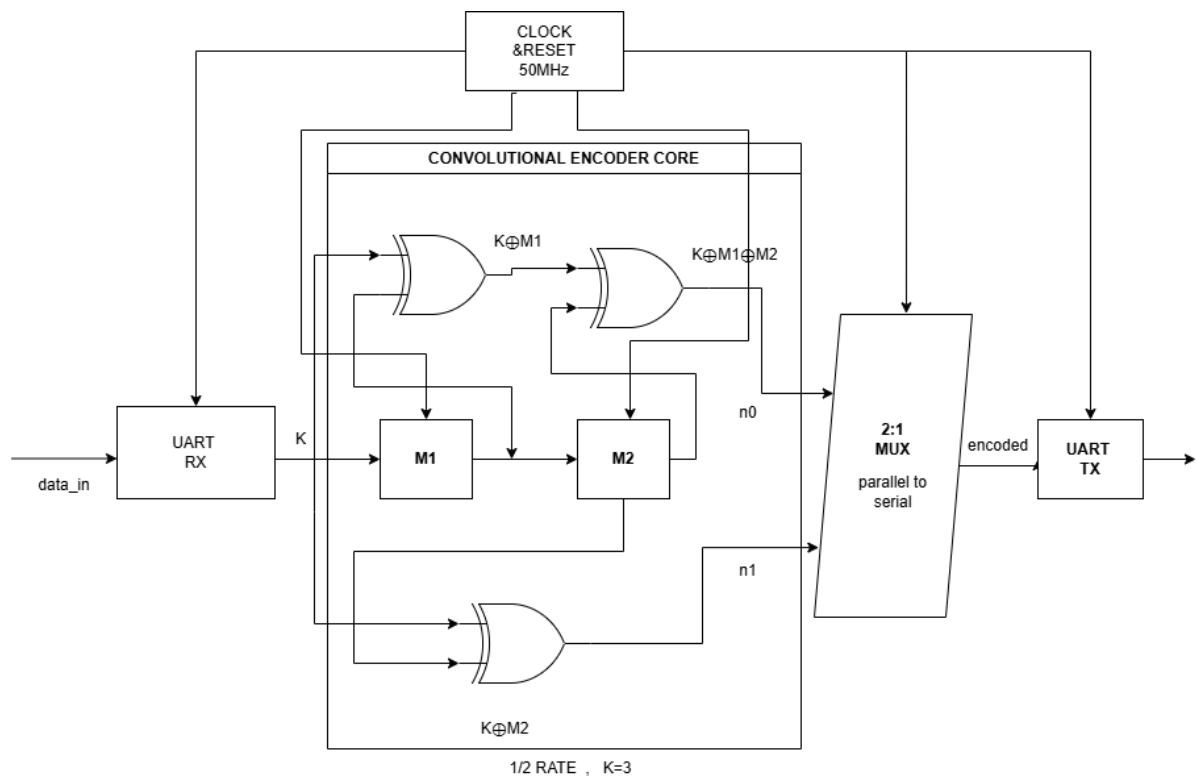
UART Interface: UART is used to receive data at 115200 bps and send encoded data at 230400 bps with proper timing to ensure reliable communication.

2:1 Output MUX: A 2:1 MUX is used to convert the two parallel output bits into a single serial output stream for transmission.

Block Diagram



Architecture Diagram



Leaf Cell Design:

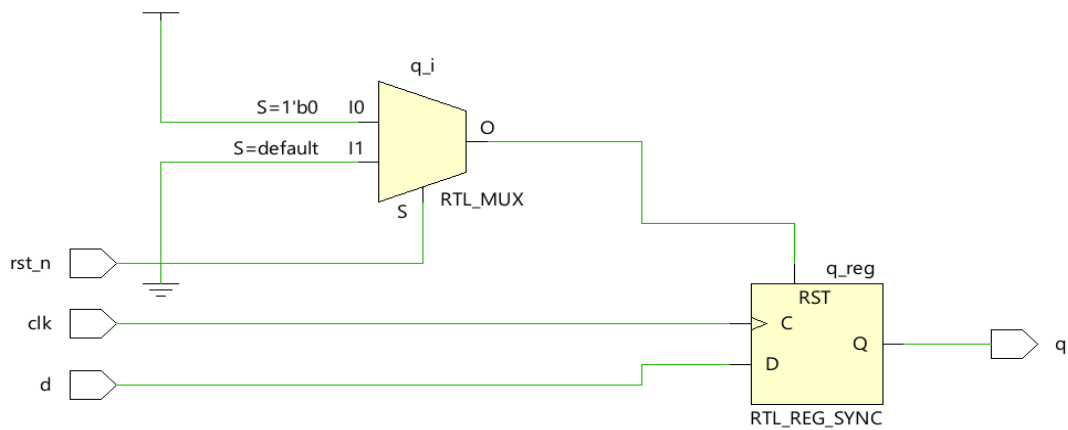
Leaf cell-1: D-flipflops

Functionality:

The D Flip-Flop stores a single bit. On every rising clock edge, it captures the value at input D and holds it at output Q until the next edge. A synchronous active-high reset sets Q to 0. The enable (EN) signal allows the D Flip-Flop to hold its current value when no new data is available.

RTL Design:

The RTL design of the D flip-flop is described using a clocked always block that is triggered on the rising edge of the clock signal. When the reset signal (rst_n) is active (logic 0), the output (q) is cleared to zero. When the reset is inactive, the flip-flop stores the input value (d) and transfers it to the output at every rising edge of the clock. This means the output follows the input with a one clock cycle delay, making the D flip-flop suitable for data storage and synchronization in sequential circuits.



Code:

```

22 module dff (
23     input clk,
24     input rst_n,    // active low reset
25     input d,
26     output reg q
27 );
28
29 always @(posedge clk) begin
30     if (!rst_n)
31         q <= 1'b0;    // reset
32     else
33         q <= d;        // store input
34 end
35
36 endmodule

```

Test bench:

```

module dff_tb();

reg clk, rst_n, d;
wire q;

// Instantiate DUT
dff uut (.clk(clk), .rst_n(rst_n), .d(d), .q(q));

// Clock generation (10ns period)
always #5 clk = ~clk;

initial begin
    $monitor("Time=%0t | clk=%b rst_n=%b d=%b q=%b", $time, clk, rst_n, d, q);

    // Initial values
    clk = 0;
    rst_n = 0;
    d = 0;

    #10 rst_n = 1;    // release reset

    #10 d = 1;
    #10 d = 0;
    #10 d = 1;
    #10 d = 1;
    #10 d = 0;

    #20 $finish;
end

endmodule

```

Optimisations Used

- Synchronous reset maps directly to FPGA hardware and avoids glitch paths.
- Enable (EN) reduces switching when no data is present.
- Single always block keeps the implementation clean and avoids extra unwanted logic.

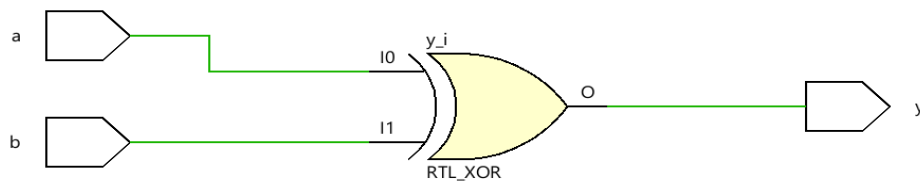
Leaf cell-2: XOR gates

Functionality: This block performs an XOR operation between M1 and M2 and sends the result to the next stage.

RTL Design:

The EXOR (exclusive OR) gate is a combinational logic circuit that produces an output based on two input signals. In the RTL design, the output is assigned using a simple logical operation where the result is high (1) only when the inputs are different and low (0) when

the inputs are the same. Since it is a combinational circuit, the output changes immediately with any change in the inputs, without the need for a clock signal. This makes the EXOR gate useful for operations like parity checking and encoding



Code:

```
module xor_gate(output y, input a, b);
    assign y = a ^ b;
endmodule
```

Test bench:

```
22 module xor_tb();
23
24     reg a, b;
25     wire y;
26
27
28     xor_gate uut (y, a, b);
29
30     initial
31     begin
32         a=1'b0; b=1'b0; #10;
33         a=1'b0; b=1'b1; #10;
34         a=1'b1; b=1'b0; #10;
35         a=1'b1; b=1'b1; #10;
36         $finish;
37     end
38
39 endmodule
```

Optimisations Used:

- Uses small XOR gates, making the design easier and efficient for FPGA implementation.

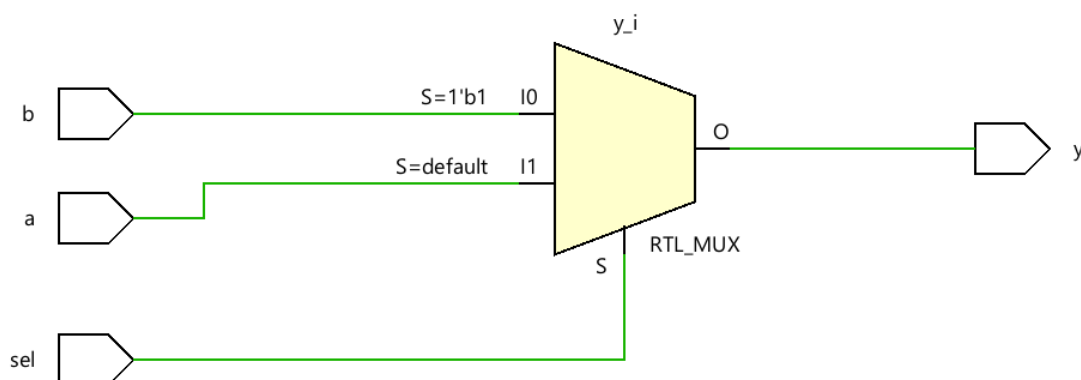
- Reuses the same input signal, so no extra hardware is required.

Leaf cell-3: 2:1 Multiplexer

Functionality: This block selects either n0 or n1 based on the control signal (sel) and sends it to the output, helping combine both signals into a single output stream.

RTL Design:

A 2:1 multiplexer is a combinational circuit that selects one of two input signals based on a control signal. In the RTL design, the output is assigned such that when the select signal (sel) is 0, the output follows input n0, and when sel is 1, the output follows input n1. The selection is done using a simple conditional (ternary) operator, making the design efficient and easy to implement. Since it is a combinational circuit, the output changes immediately with changes in inputs or the select signal.



Code:

```

22 module mux2to1 (
23     input a, b,      // inputs
24     input sel,       // select line
25     output y         // output
26 );
27
28 assign y = (sel) ? b : a;
29
30 endmodule

```

Test bench:

```

module mux2to1_tb();

reg a, b, sel;
wire y;

// Instantiate DUT
mux2to1 uut (.a(a), .b(b), .sel(sel), .y(y));

initial begin
    $monitor("a=%b b=%b sel=%b y=%b", a, b, sel, y);

    a=0; b=0; sel=0; #10;
    a=0; b=1; sel=0; #10;
    a=1; b=0; sel=0; #10;
    a=1; b=1; sel=0; #10;

    a=0; b=0; sel=1; #10;
    a=0; b=1; sel=1; #10;
    a=1; b=0; sel=1; #10;
    a=1; b=1; sel=1; #10;

    $finish;
end

endmodule

```

Optimisations Used:

- Uses a simple assign statement, so only one logic unit is needed and delay is low.
- Uses minimum possible hardware, as a 2:1 MUX only needs one logic unit.

Input test Cases:

D-flipflop 1

- When reset is 0, output is forced to 0
- When reset is released, output follows input on clock edge
- The flip-flop stores the input with one clock delay

D-flip-flop 2

- When reset is 0, output m2 is forced to 0
- After reset, m2 follows m1 on every clock edge

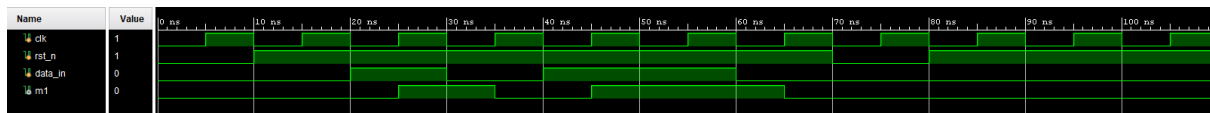
XOR-gate

Time (ns)	m1	m2	Expected Output	Description
0	0	0	0	Both inputs same → output 0

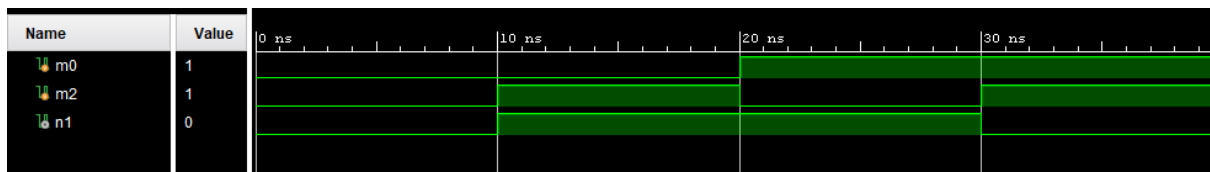
10	0	1	1	Inputs different → output 1
20	1	0	1	Inputs different → output 1
30	1	1	0	Both inputs same → output 0

Leaf Cell Simulation

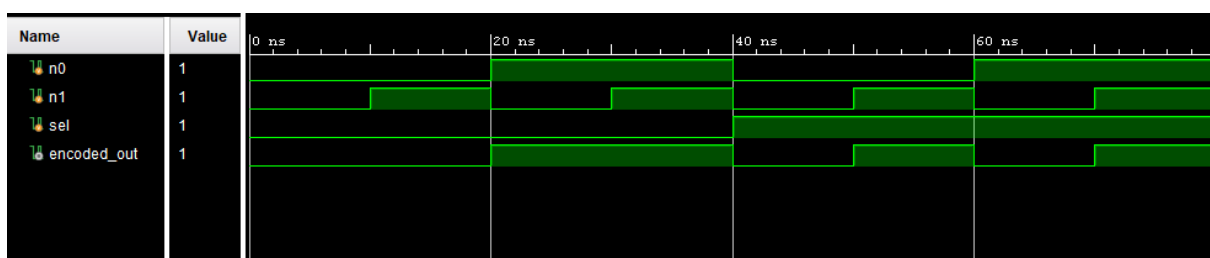
D Flip-Flop



XOR gate



2:1 MUX



Appendix

Tool Used: Xilinx Vivado

References:

- [1] Y. Sun and Z. Ding, "FPGA Design and Implementation of a Convolutional Encoder and a Viterbi Decoder Based on 802.11a for OFDM," *Wireless Engineering and Technology*, vol. 3, pp. 125–131, 2012.
- [2] Y. S. Wong, W. J. Ong, J. H. Chong, C. K. Ng, and N. K. Noordin, "Implementation of Convolutional Encoder and Viterbi Decoder using VHDL," in *Proc. Student Conf. on Research and Development (SCORed)*, 2009, pp. 22–25.
- [3] V. Kavinilavu, S. Salivahanan, V. S. K. Bhaaskaran, S. Sakthikumaran, B. Brindha, and C. Vinoth, "Implementation of Convolutional Encoder and Viterbi Decoder using Verilog HDL," in *Proc. Int. Conf. on Electronics Computer Technology (ICECT)*, 2011, pp. 297–300.
- [4] B. Soreng and S. Kumar, "Efficient Implementation of Convolution Encoder and Viterbi Decoder," in *Proc. Int. Conf. on Circuits, Power and Computing Technologies (ICCPCT)*, 2013, pp. 1270–1273.